

Queue Data Structures

- **Queue:**

- A Queue, also known as **First in First Out (FIFO)** system, is a linear line in which deletions can take place only at one end of the list, the “**front**” of the list, and insertion can take place only at the other end of list, the “**rear**” of the list [1].

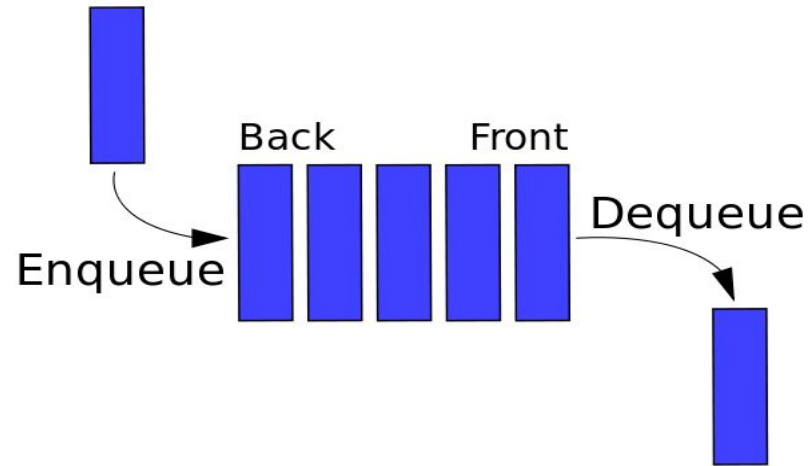


Overview of different Data Structures



- **Queue:**

- A Queue, also known as **First in First Out (FIFO)** system, is a linear line in which deletions can take place only at one end of the list, the “**front**” of the list, and insertion can take place only at the other end of list, the “**rear (back)**” of the list [1], [2].





Queue Data Structures

- Array representation of Queue:

```
#define MAX 100
```

```
struct Queue
```

```
{
```

```
    int front, rear;
```

```
    int items[MAX];
```

```
}
```

```
struct Queue q;
```

```
q. front=0, q. rear=0;
```

Basic Operation in Queue



- **Insertion: Enqueue Operation**
- **Deletion: Dequeue Operation**



Types of Queue

- ◆ Circular Queue
- ◆ DEQUEUE
 - ◆ Input Restricted DEQUEUE
 - ◆ Output Restricted DEQUEUE
- ◆ Priority Queue



Basic Operation in Queue

• Insertion: Enqueue Operation [1]

QINSERT(Queue, N, FRONT, REAR, ITEM)

This procedure inserts an element ITEM into a queue.

1. [Queue already filled?]

If $\text{FRONT} = 1$ and $\text{REAR} = N$, or if $\text{FRONT} = \text{REAR} + 1$, then:

Write: **OVERFLOW**, and Return.



Basic Operation in Queue

• Insertion: Enqueue Operation [1]

2. [Find new value of REAR.]
If $FRONT := NULL$, then: [Queue initially empty.]
 Set $FRONT := 1$ and $REAR := 1$.
Else if $REAR = N$, then:
 Set $REAR := 1$.
Else:
 Set $REAR := REAR + 1$.
[End of If structure.]
3. Set $QUEUE[REAR] := ITEM$. [This inserts new element.]
4. Return.



Basic Operation in Queue

- **Deletion: Dequeue Operation [1]**

QDELETE(Queue, N, FRONT, REAR, ITEM)

This procedure deletes an element from a queue and assigns it to the variable ITEM.

1. [Queue already empty?]

If $FRONT := NULL$, then: Write: UNDERFLOW, and Return.

2. Set $ITEM := QUEUE[FRONT]$.



Basic Operation in Queue

• Deletion: Dequeue Operation [1]

3. [Find new value of FRONT.]

If $FRONT = REAR$, then: [Queue has only one element to start.]

Set $FRONT := NULL$ and $REAR := NULL$.

Else if $FRONT = N$, then:

Set $FRONT := 1$.

Else:

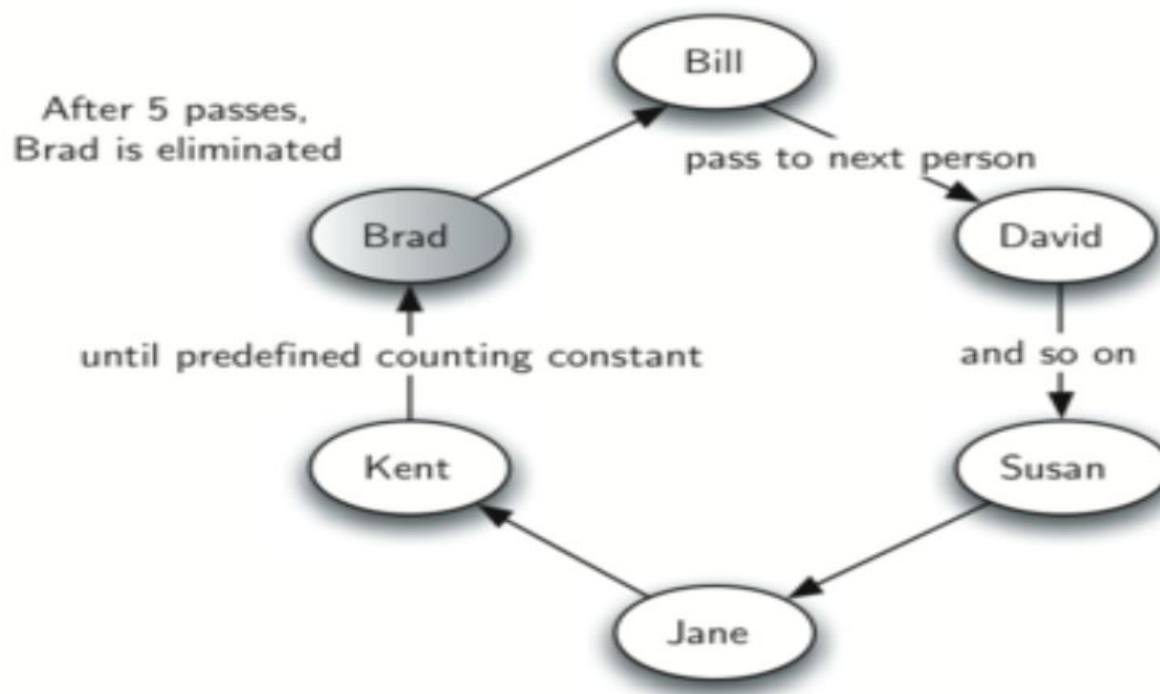
Set $FRONT := FRONT + 1$.

[End of If structure.]

4. Return.

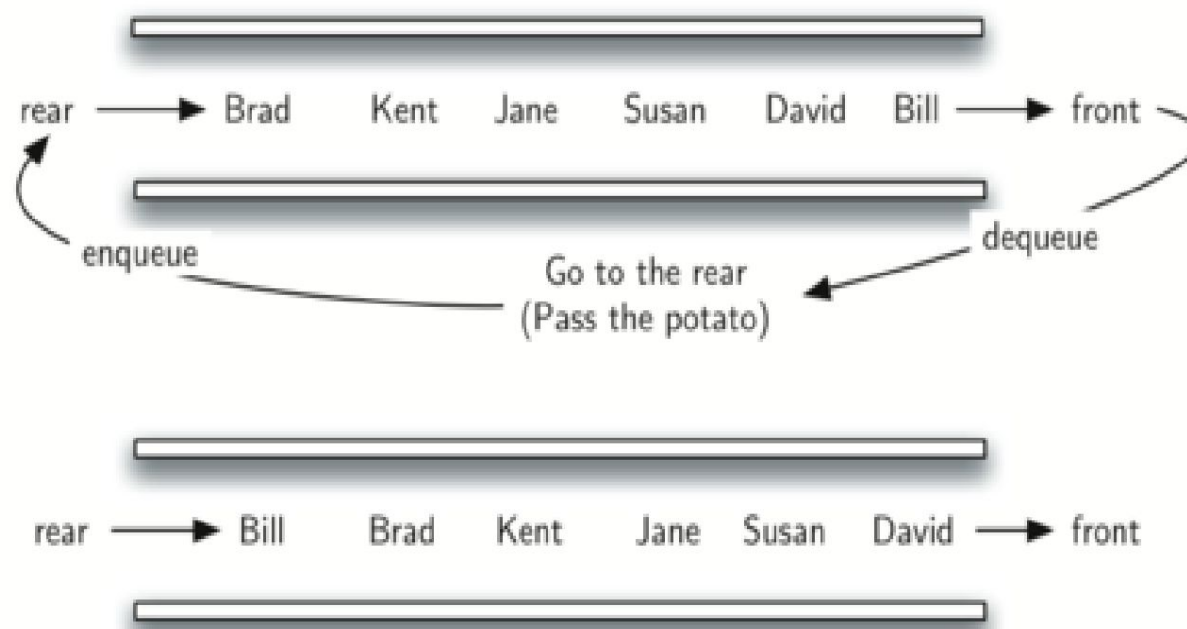
Problems based on Queue

- **Hot Potato Game:** This game is a modern-day equivalent of the famous Josephus problem [3].



Problems based on Queue

- **Hot Potato Game:** This game is a modern-day equivalent of the famous Josephus problem [3].





Problems based on Queue

- **Palindrome Checker [3]:**

- An interesting problem that can be easily solved using the **deque** data structure is the classic **palindrome problem**.
- A **palindrome** is a string that reads the same forward and backward, for example, radar, toot, and madam.



Recursion

- Many objects in Mathematics are defined by presenting a process to produce that object.
- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function.
- Examples:
 - Factorial Function
 - Multiplication of Natural Numbers
 - Fibonacci series
 - Binary Search
 - Tower of Hanoi Problem



Problems based on Queue

- Problem: **Sliding Window Maximum**
- **Arr[1,.....n]**
- **K= sliding window size**
- **Find the maximum in each slide window**



References

1. Seymour Lipschutz, “Data Structures”, Schaum’s Series McGraw Hill edition 2013.
2. <https://en.wikipedia.org/>
3. **Problem Solving with Algorithms and Data Structures**
Release 3.0 Brad Miller, David Ranum